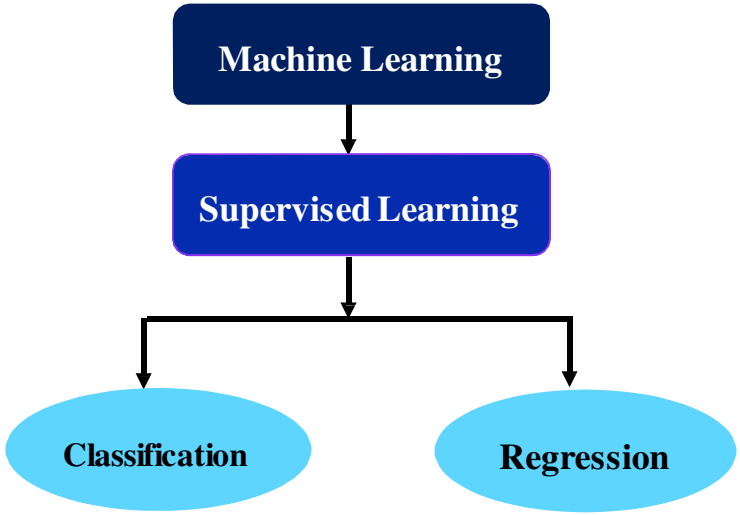
Unit –II Classification & Regression

Unit–II:	No. of Lectures: 09 Hours	Marks: 12
Classification / Regression: Classifications, decision tree learning, naive bayes, linear regression, logistic regression, Linear regression models, support vector machine, beyond binary classifications: multiclass or multinomial classification.		

Classification and Regression in Machine Learning

- Data scientists use many different kinds of machine learning algorithms to discover patterns in big data that lead to actionable insights.
- At a high level, these different algorithms can be classified into two groups based on the way they "learn" about data to make predictions:
  - Supervised learning
  - Unsupervised learning.

Classification and Regression in Machine Learning



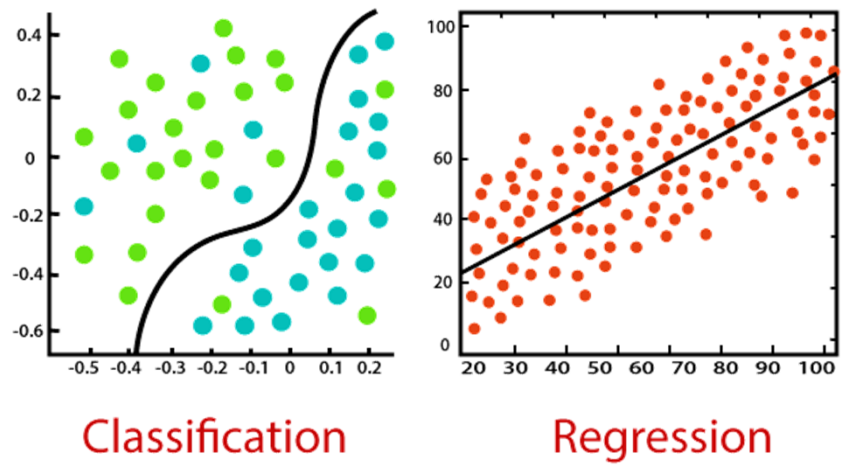
- Classification is a type of **supervised learning.**
- It specifies the class to which data elements belong to and is best used when the output has finite and discrete values.
- It predicts a class for an input variable as well.

Classification and Regression in Machine Learning

- Supervised learning requires that the data used to train the algorithm is already labeled with correct answers.
- For example, a classification algorithm will learn to identify animals after being trained on a dataset of images that are properly labeled with the species of the animal and some identifying characteristics.
- Supervised learning problems can be further grouped into **Regression** and **Classification** problems.
- Both problems have as goal the construction of a brief model that can predict the value of the dependent attribute from the attribute variables.
- The difference between the two tasks is the fact that the dependent attribute is **numerical for regression** and **categorical for classification.**

Classification and Regression in Machine Learning

The main difference between Regression and Classification algorithms that Regression algorithms are used to **predict the continuous** values such as price, salary, age, etc. and Classification algorithms are used to **predict/Classify the discrete values** such as Male or Female, True or False, Spam or Not Spam



Classification in Machine Learning

- A classification problem is when the output variable is a category, such as “apple” or “mango” or “yes” and “no”. A classification model attempts to draw some conclusion from observed values.
- Given one or more inputs a classification model will try to predict the value of one or more outcomes.
- For example, when filtering emails “spam” or “not spam”, when looking at transaction data, “fraudulent”, or “authorized”.
- In short Classification either predicts categorical class labels or classifies data (construct a model) based on the training set and the values (class labels) in classifying attributes and uses it in classifying new data.
- There are a number of classification models. Classification models include logistic regression, decision tree, random forest, SVM, one-vs-rest, and Naive Bayes.

Classification in Machine Learning

- **For example: Which of the following is/are classification problem(s)?**
- Predicting house price based on area
- Predicting whether monsoon will be normal next year
- Predict the number of copies a music album will be sold next month

Classification in Machine Learning

- **Classification** is the process of finding or discovering a model or function which helps in separating the data into multiple categorical classes i.e. discrete values.
- In classification, data is categorized under different labels according to some parameters given in input and then the labels are predicted for the data.
- The derived mapping function could be demonstrated in the form of "IF-THEN" rules.
- The classification process deal with the problems where the data can be divided into binary or multiple discrete labels.
- Let's take an example, suppose we want to predict the possibility of the wining of match by Team **A** on the basis of some parameters recorded earlier. Then there would be two labels **Yes** and **No**.

Classification in Machine Learning

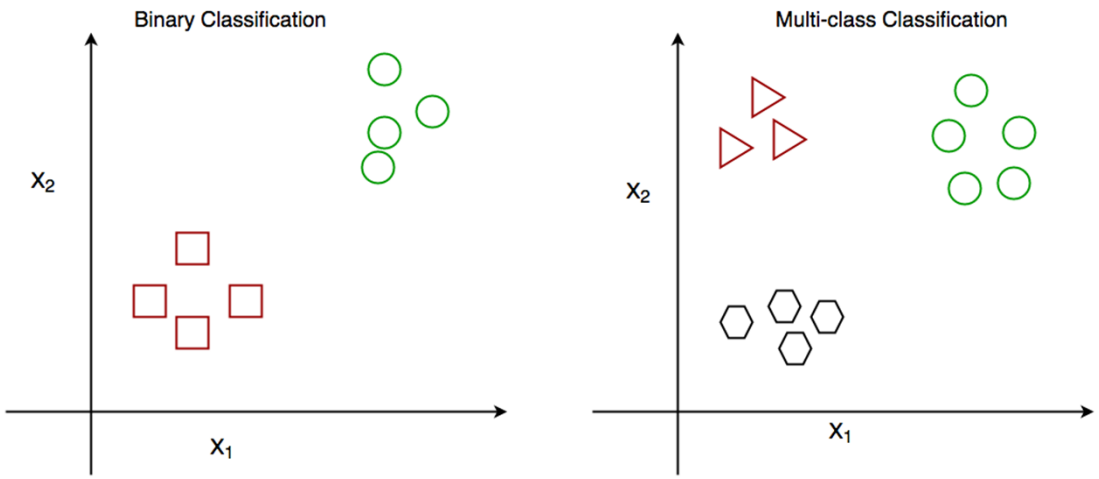


Fig : Binary Classification and Multiclass Classification

Classification Algorithms in Machine Learning

- Decision Tree Classification
- Naïve Bayes
- Logistic Regression
- Support Vector Machines
- Random Forest Classification

Regression in Machine Learning

- **Regression** is the process of finding a model or function for distinguishing the data into continuous real values instead of using classes or discrete values.
- It can also identify the distribution movement depending on the historical data.
- Because a regression predictive model predicts a quantity, therefore, the skill of the model must be reported as an error in those predictions.
- Let's take a example in regression also, where we are finding the possibility of rain in some particular regions with the help of some parameters recorded earlier.
- Then there is a probability associated with the rain.

Regression in Machine Learning

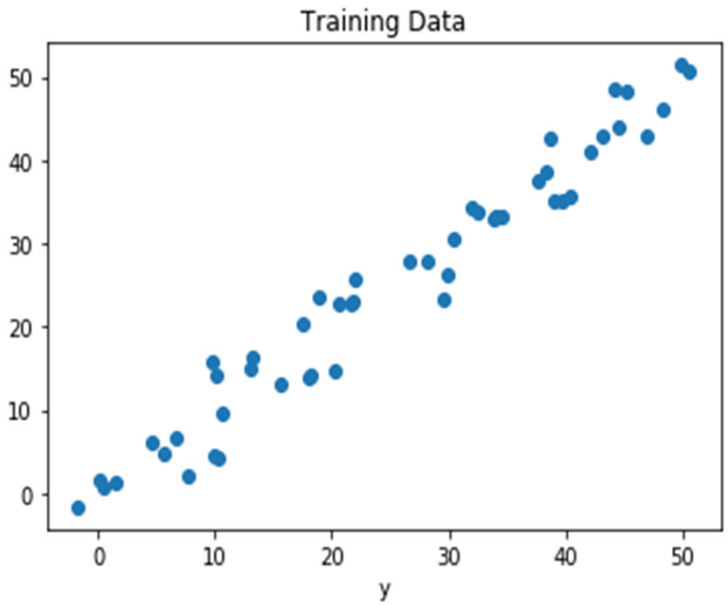


Fig : Regression of Day vs Rainfall (in mm)

Regression in Machine Learning

- A regression problem is when the output variable is a real or continuous value.
- Many different models can be used, the simplest is the **linear regression**.
- It tries to fit data with the best hyper-plane which goes through the points.
- **For Examples: Which of the following is a regression task?**
- Predicting age of a person
- Predicting nationality of a person
- Predicting whether stock price of a company will increase tomorrow

Regression Algorithm in Machine Learning

- Simple Linear Regression
- Multiple Linear Regression
- Polynomial Regression
- Support Vector Regression
- Decision Tree Regression
- Random Forest Regression

PARAMENTER	CLASSIFICATION	REGRESSION
Basic	Mapping Function is used for mapping of values to predefined classes.	Mapping Function is used for mapping of values to continuous output.
Involves prediction of	Discrete values	Continuous values or real values
Nature of the predicted data	Unordered	Ordered
Method of calculation	by measuring accuracy	by measurement of root mean square error
Algorithms	Decision tree, logistic regression, etc.	Regression tree (Random forest), Linear regression, etc.
Output	Try to find the decision boundary, which can divide the dataset into different classes	Try to find the best fit line, which can predict the output more accurately.
Example	Classification Algorithms can be used to solve classification problems such as Identification of spam emails, Speech Recognition, Identification of cancer cells, etc.	Regression algorithms can be used to solve the regression problems such as Weather Prediction, House price prediction, etc.
Types	The Classification algorithms can be divided into Binary Classifier and Multi-class Classifier.	The regression Algorithm can be further divided into Linear and Non-linear Regression.

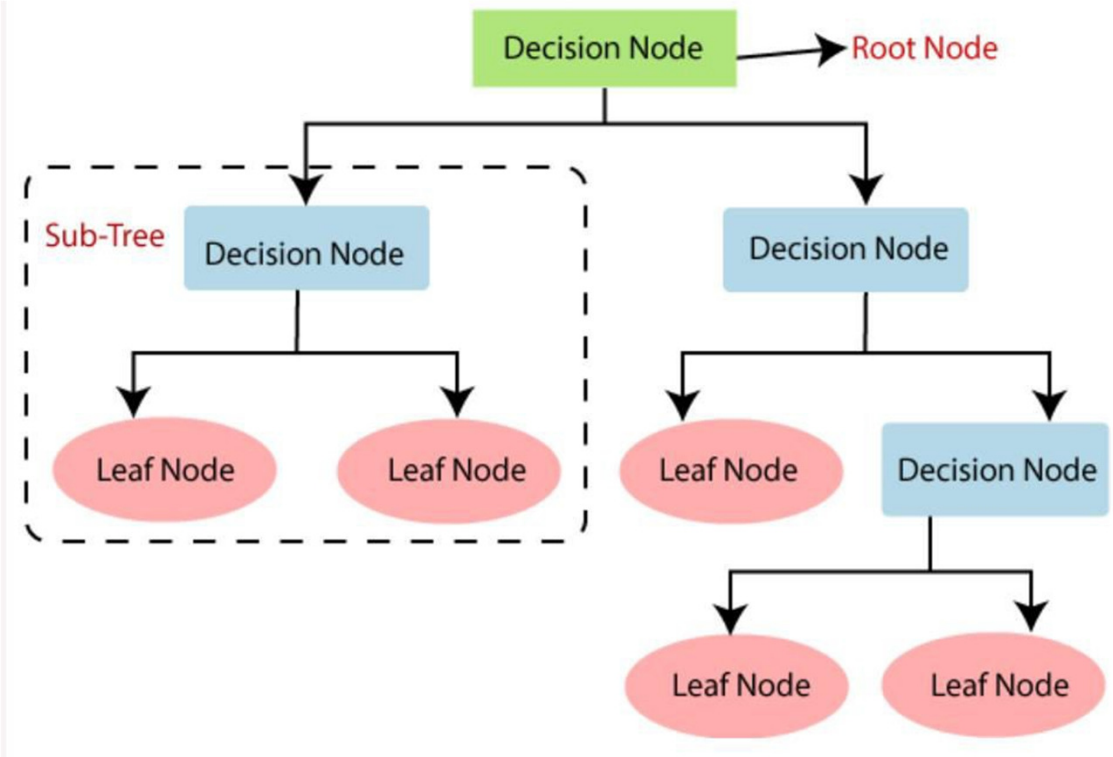
Machine Learning Algorithms

- Decision Tree
- Naïve Bayes
- Linear Regression
- Logistic Regression
- Support Vector Machines

Decision Tree Learning

- A **decision tree** is a decision support tool that uses a tree-like graph or model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility.
- It is one way to display an algorithm that only contains conditional control statements.
- A decision tree is a flowchart-like structure in which
  - each **internal node** (decision node) represents a "test" on an attribute (e.g. whether a coin flip comes up heads or tails),
  - each **branch** represents the outcome of the test,
  - each **leaf node** represents a class label (decision taken after computing all attributes).
- The paths from root to leaf represent **classification rules**.

Decision Tree



Decision Tree

- Tree based learning algorithms are considered to be one of the best and mostly used supervised learning methods.
- Tree based methods empower predictive models with high accuracy, stability and ease of interpretation.
- Unlike linear models, they map non-linear relationships quite well.
- They are adaptable at solving any kind of problem (classification or regression).
- Decision Tree algorithms are referred to as **CART (Classification and Regression Trees)**.

Terminologies in Decision Tree Learning

•**Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

•**Decision Node:** When a sub-node splits into further sub-nodes, then it is called decision node.

•**Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

•**Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

•**Branch/Sub Tree:** A tree formed by splitting the tree.

•**Pruning:** Pruning is the process of removing the unwanted branches from the tree.

•**Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes

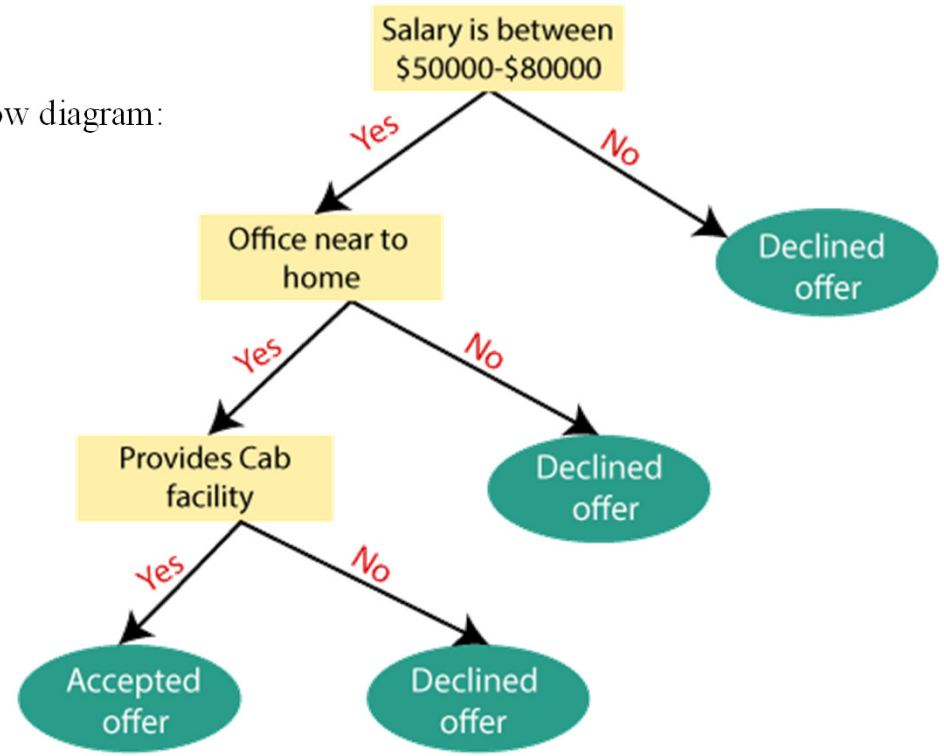
Example 1

- **Example:** Suppose there is a candidate who has a job offer and wants to decide whether he/she should accept the offer or Not.

  So, to solve this problem, the decision tree starts with the root node (Salary attribute by **Attribute Selection Measure (ASM).**
- The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels.
- The next decision node further gets split into one decision node (Cab facility) and one leaf node.
- Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer).

Example 1

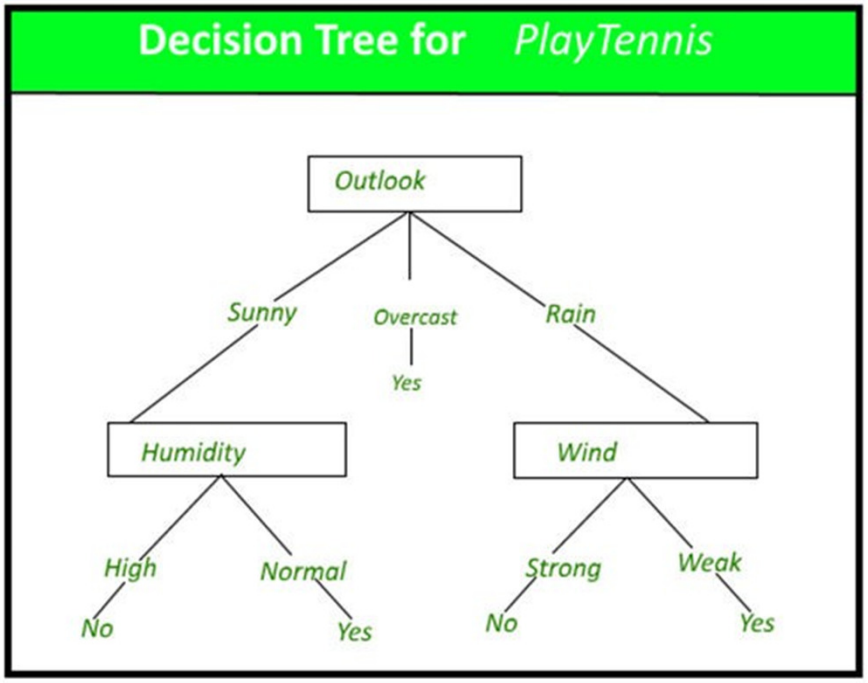
- Consider the below diagram:



Example 2

- Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the **classification of the instance**.
- An instance is classified by starting at the root node of the tree, testing the attribute specified by this node, then moving down the tree branch corresponding to the value of the attribute as shown in the figure.
- This process is then repeated for the subtree rooted at the new node.

Example 2


Decision Tree for PlayTennis
Outlook
Sunny
Overcast
Rain
Yes
Humidity
Wind
High
Normal
Strong
Weak
No
Yes
No
Yes

Example 2

The decision tree in above figure classifies a particular morning, according to whether it is suitable for playing tennis and returning the classification associated with the particular leaf. (in this case Yes or No).

For example, the instance

(Outlook = Sunny, Humidity = High)

would be sorted down the leftmost branch of this decision tree and would therefore be classified as a negative instance.

How does the Decision Tree algorithm Work?

- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree.
- This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.
- For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further.
- It continues the process until it reaches the leaf node of the tree.

Decision Tree algorithm

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Attribute Selection Measures

- While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes.
- So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM.**
- By this measurement, user can easily select the best attribute for the nodes of the tree.
- There are two popular techniques for ASM, which are:
  - **Information Gain**
  - **Gini Index**

1. Information Gain

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)]

1. Information Gain

Entropy:

- Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data.
- Entropy can be calculated as:

$$\text{Entropy}(s) = -p \log_2 p - q \log_2 q$$

- Where,
- S = Total number of samples
- p = probability of yes
- q = probability of no

2. Gini Index

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:

$$\textbf{Gini Index} = 1 - \sum P_j^2$$

Example

- A decision node (e.g., Outlook) has two or more branches (e.g., Sunny, Overcast and Rainy).
- Leaf node (e.g., Play) represents a classification or decision.
- The topmost decision node in a tree which corresponds to the best predictor called **root node**.
- The core algorithm for building decision trees called **ID3** by J. R. Quinlan which employs a top-down, greedy search through the space of possible branches with no backtracking.
- ID3 uses ***Entropy*** and ***Information Gain*** to construct a decision tree.

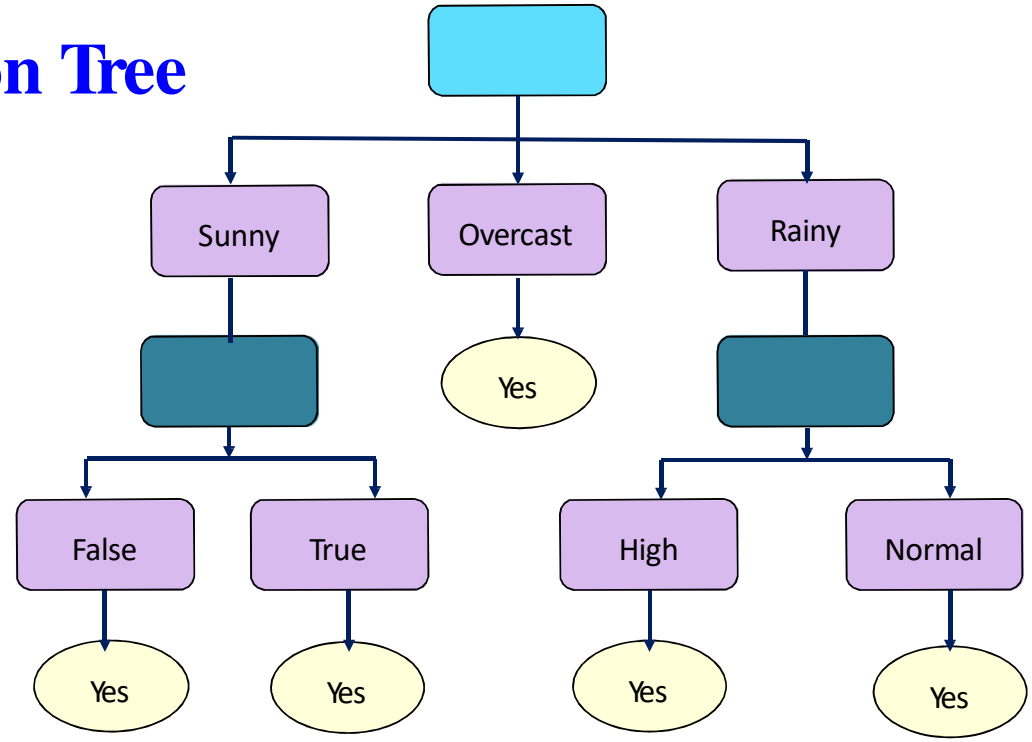
Example

Predictors

Target

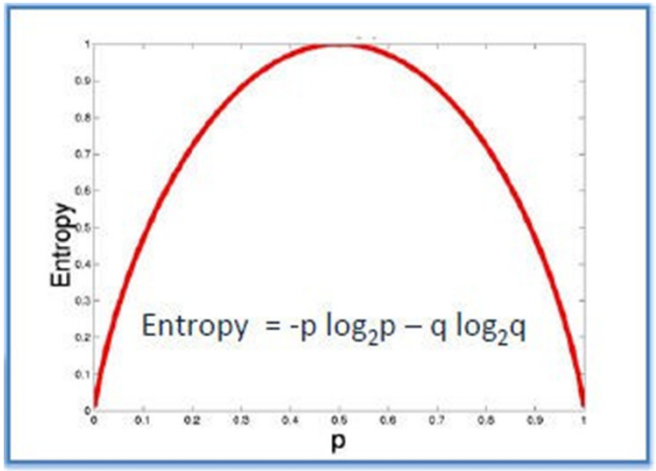
Outlook	Temp	Humidity	Wind	Play Golf
Rainy	Hot	High	False	*No*
Rainy	Hot	High	True	*No*
Overcast	Hot	High	False	*Yes*
Sunny	Mild	High	False	*Yes*
Sunny	Cool	Normal	False	*Yes*
Sunny	Cool	Normal	True	*No*
Overcast	Cool	Normal	True	*Yes*
Rainy	Mild	High	False	*No*
Rainy	Cool	Normal	False	*Yes*
Sunny	Mild	Normal	False	*Yes*
Rainy	Mild	Normal	True	*Yes*
Overcast	Mild	High	True	*Yes*
Overcast	Hot	Normal	False	*Yes*
Sunny	Mild	High	True	*No*

Decision Tree



Example

- **Entropy**
- A decision tree is built top-down from a root node and involves partitioning the data into subsets that contain instances with similar values (homogenous).
- ID3 algorithm uses entropy to calculate the homogeneity of a sample.
- If the sample is completely homogeneous the entropy is **zero** and if the sample is an equally divided it has entropy of **one**.



Entropy = $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$

Example

- To build a decision tree, we need to calculate two types of entropy using frequency tables as follows:

a) Entropy using the frequency table of one attribute:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Total no. of occurrences are 14 out of which 5 are for class '*No*' and 9 are for class '*Yes*'.

Play Golf	
Yes	No
9	5

Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

=Entropy(5/14, 9/14)

=Entropy(0.36, 0.64)

=-(0.36 $\log_2$ 0.36)-(0.64 $\log_2$ 0.64)

=0.53+0.41

= 0.94

Example

b) Entropy using the frequency table of two attributes:

Entropy(*Two attribute*)=(*WeightedAvg*) *Entropy(*each attribute*)

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

Entropy(*Sunny*)= E(3,2) = - (3/5) $\log_2$(3/5) - (2/5) $\log_2$(2/5)
= - (0.6) $\log_2$(0.6) - (0.4) $\log_2$ (0.4)
= 0.44+0.53
= 0.97

Entropy(*Overcast*) = E(4,0) = - (4/4) $\log_2$(4/4) - (0/4) $\log_2$(0/4)
= - (1) $\log_2$(1) - (0) $\log_2$ (0)
= 0.0

Entropy(*Rainy*)= E(3,2) = - (2/5) $\log_2$(2/5) - (3/5) $\log_2$(3/5)
= - (0.4) $\log_2$ (0.4) - (0.6) $\log_2$(0.6)
= 0.53+0.44
= 0.97

E(PlayGolf, Outlook) = P(Sunny)*E(3,2) + P(Overcast)*E(4,0) + P(Rainy)*E(2,3)

= (5/14)*0.971 + (4/14)*0.0 + (5/14)*0.971

= 0.693

Example

Information Gain:

- The information gain is based on the decrease in entropy after a dataset is split on an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain (i.e., the most homogeneous branches).
- ***Step 1*:** Calculate entropy of the target.
- ***Step 2***: The dataset is then split on the different attributes. The entropy for each branch is calculated. Then it is added proportionally, to get total entropy for the split. The resulting entropy is subtracted from the entropy before the split. The result is the Information Gain, or decrease in entropy.

Information Gain(G) = Entropy(*play Golf*) - Entropy(*Play Golf, Outlook*)

Example

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

$$Gain(T,X) = Entropy(T) - Entropy(T,X)$$

G(PlayGolf, Outlook) = **E**(PlayGolf) – **E**(PlayGolf, Outlook)

= 0.940 – 0.693 = 0.247

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

Example

- ***Step 3*:** Choose attribute with the largest information gain as the decision node, divide the dataset by its branches and repeat the same process on every branch.

★		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

Outlook	Play Golf
Sunny	Yes
	Yes
	No
	Yes
	No

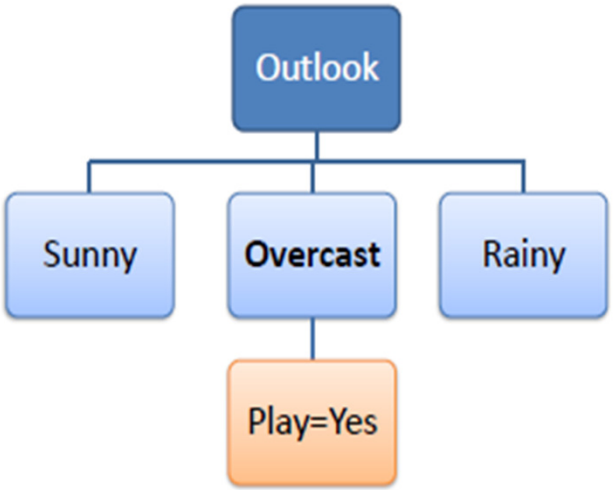
Outlook	Play Golf
Overcast	Yes
	Yes
	Yes
	Yes

Outlook	Play Golf
Rainy	No
	No
	No
	No
	Yes

Example

- ***Step 4a***: A branch with entropy of 0 is a leaf node.

Temp	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes

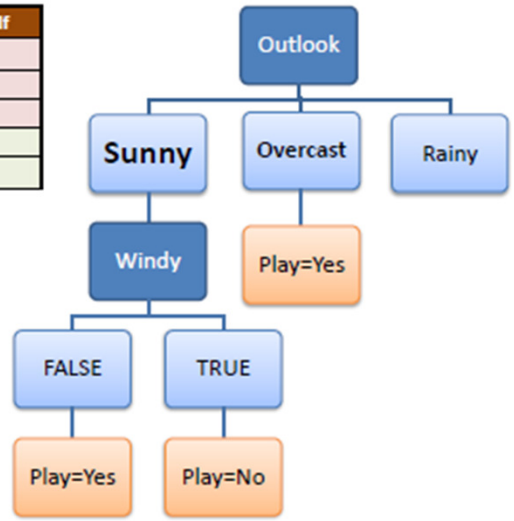


Entropy(*Overcast*) = E(4,0) = 0.0

Example

- *Step 4b*: A branch with entropy more than 0 needs further splitting

Temp	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



- *Step 5*: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified

Decision Tree to Decision Rules

- A decision tree can easily be transformed to a set of rules by mapping from the root node to the leaf nodes one by one.

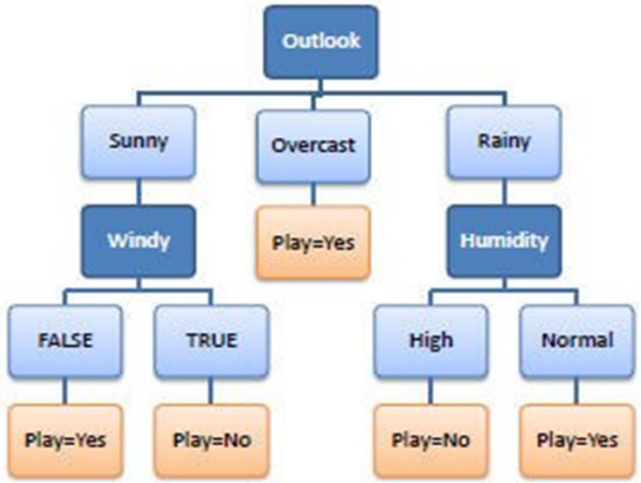
R_1: IF (Outlook=Sunny) AND (Windy=FALSE) THEN Play=Yes

R_2: IF (Outlook=Sunny) AND (Windy=TRUE) THEN Play=No

R_3: IF (Outlook=Overcast) THEN Play=Yes

R_4: IF (Outlook=Rainy) AND (Humidity=High) THEN Play=No

R_5: IF (Outlook=Rain) AND (Humidity=Normal) THEN Play=Yes



Types of Decision Trees

Types of decision tree is based on the type of target variable that user have. It can be of two types:

- **Categorical Variable Decision Tree:** Decision Tree which has categorical target variable then it called as categorical variable decision tree. E.g.:- In above scenario of student problem, where the target variable was "Student will play Golf or not" i.e. YES or NO.
- **Continuous Variable Decision Tree:** Decision Tree has continuous target variable then it is called as Continuous Variable Decision Tree.

Advantages of Decision Tree

- Easy to Understand
- Useful in Data exploration
- Decision trees implicitly perform variable screening or feature selection.
- Decision trees require relatively little effort from users for data preparation.
- Less data cleaning required
- Data type is not a constraint
- Non-Parametric Method
- Non-linear relationships between parameters do not affect tree performance.

Disadvantages of Decision Tree

- Over fitting
- Not fit for continuous variables
- Calculations can become complex when there are many class label.
- Generally, it gives low prediction accuracy for a dataset as compared to other machine learning algorithms.
- Information gain in a decision tree with categorical variables gives a biased response for attributes with greater no. of categories.

Applications of Decision Tree

- Direct Marketing
- Customer Retention
- Fraud Detection
- Diagnosis of Medical Problems

Machine Learning Algorithms

- Decision Tree
- **Naïve Bayes**
- Linear Regression
- Logistic Regression
- Support Vector Machines

Naïve Bayes

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems.
- It is mainly used in *text classification* that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.

Naïve Bayes

- Naïve Bayes technique which makes a True assumption that all the predictors are independent to each other.
- In simple words, the assumption is that the presence of a feature in a class is independent to the presence of any other feature in the same class.
- For example, a phone may be considered as smart if it is having touch screen, internet facility, good camera etc. Though all these features are dependent on each other, they contribute independently to the probability of that the phone is a smart phone.
- Some popular examples of Naïve Bayes Algorithm are spam filtration, Sentimental analysis, and classifying articles.

Bayes' Theorem

- In Bayesian classification, the main interest is to find the posterior probabilities $P(A|B)$, from $P(A)$, $P(B)$, and $P(B|A)$. Naive Bayes classifier assume that the effect of the value of a predictor (B) on a given class (A)is independent of the values of other predictors. This assumption is called class conditional independence.

- With the help of Bayes theorem, we can express this in quantitative form as follows:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- Here, (A | B) is the posterior probability of *class* A (target) given *predictor* B(feature)
- P(A) is the prior probability of *class*.
- P(B|A) is the likelihood which is the probability of *predictor* given class.
- P(B) is the prior probability of *predictor*.

Example: Naïve Bayes

Now, with regards to outlook dataset, we can apply Bayes' theorem in following way:

Likelihood

Class Prior Probability

$$P(c \mid x) = \frac{P(x \mid c)P(c)}{P(x)}$$

Posterior Probability

Predictor Prior Probability

$$P(c \mid \mathrm{X}) = P(x_1 \mid c) \times P(x_2 \mid c) \times \cdots \times P(x_n \mid c) \times P(c)$$

where, 'c' is class variable and 'x' is a dependent feature vector (of size n)

Example: Naïve Bayes

Total no. of samples for class 1:
Play_golf ="Yes"= 9

Total no. of samples for class 2:
Play_golf ="No"= 5

Predictors				Target
Rainy	Hot	High	False	*No*
Rainy	Hot	High	True	*No*
Overcast	Hot	High	False	*Yes*
Sunny	Mild	High	False	*Yes*
Sunny	Cool	Normal	False	*Yes*
Sunny	Cool	Normal	True	*No*
Overcast	Cool	Normal	True	*Yes*
Rainy	Mild	High	False	*No*
Rainy	Cool	Normal	False	*Yes*
Sunny	Mild	Normal	False	*Yes*
Rainy	Mild	Normal	True	*Yes*
Overcast	Mild	High	True	*Yes*
Overcast	Hot	Normal	False	*Yes*
Sunny	Mild	High	True	*No*

Example: Naïve Bayes

For **data sample X = (Outlook= rainy, Temp= cool, Humidity= high, Windy= true)**

- P(Outlook = rainy | play_golf="Yes")= 2/9=0.222
- P(Outlook = rainy | play_golf="No")= 3/5=0.6
- P(Temp = cool | play_golf="Yes")= 3/9=0.333
- P(Temp = cool | play_golf="No")= 1/5=0.2
- P(Humidity= high | play_golf="Yes")= 3/9=0.333
- P(Humidity = high | play_golf="No")= 4/5=0.8
- P(Windy= true | play_golf="Yes")= 3/9=0.333
- P(Windy = true | play_golf="No")= 3/5=0.6

Likelihood Table

		Play Golf	
		Yes	No
Outlook	Sunny	3/9	2/5
	Overcast	4/9	0/5
	Rainy	2/9	3/5

		Play Golf	
		Yes	No
Humidity	High	3/9	4/5
	Normal	6/9	1/5

		Play Golf	
		Yes	No
Temp.	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5

		Play Golf	
		Yes	No
Windy	False	6/9	2/5
	True	3/9	3/5

Example: Naïve Bayes

- $P(x|c) = P(x \mid play_golf = \text{“Yes”})$

  $= 0.222 \text{ X } 0.333 \text{ X } 0.333 \text{ X } 0.333$

  $= 0.0081$

- $P(x|c) = P(x \mid play_golf = \text{“No”})$

  $= 0.6 \text{ X } 0.2 \text{ X } 0.8 \text{ X } 0.6$

  $= 0.0567$

Example: Naïve Bayes

- Total no. of sample for class "Yes"= 9/14 = 0.64
- Total no. of sample for class "No"= 5/14 = 0.36
- P(x |c) * P(c)= P(x | play_golf= "Yes") * P(play_golf= "Yes")

  = 0.0081 X 0.64

  =0.0051
- P(x |c) * P(c)= P(x | play_golf= "No") * P(play_golf= "No")

  = 0.0567 X 0.36

  =0.020

X data sample belongs to Play a golf = No

Types of Naïve Bayes

There are three types of Naive Bayes:

1.Gaussian: The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.

2.Multinomial: The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc. The classifier uses the frequency of words for the predictors.

Types of Naïve Bayes

3. Bernoulli: The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

Advantages of Naïve Bayes Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It is the most popular choice for **text classification problems**.
- When assumption of independent predictors holds true, a Naive Bayes classifier performs better as compared to other models.
- Naive Bayes requires a small amount of training data to estimate the test data. So, the training period is less.

Disadvantages of Naïve Bayes Classifier:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.
- Main imitation of Naive Bayes is the **assumption of independent predictors**. Naive Bayes implicitly assumes that all the attributes are mutually independent. In real life, it is almost impossible that we get a set of predictors which are completely independent.
- If categorical variable has a category in test data set, which was not observed in training data set, then model will assign a 0 (zero) probability and will be unable to make a prediction. This is often known as **Zero Frequency**.

Application of Naïve Bayes Classifier

- **Real time Prediction:** Naive Bayes is an eager learning classifier and it is sure fast. Thus, it could be used for making predictions in real time.
- **Multi class Prediction:** This algorithm is also well known for multi class prediction feature. It is able to predict the probability of multiple classes of target variable.
- **Text classification/ Spam Filtering/ Sentiment Analysis:** Naive Bayes classifiers mostly used in text classification (due to better result in multi class problems and independence rule) have higher success rate as compared to other algorithms. As a result, it is widely used in Spam filtering (identify spam e-mail) and Sentiment Analysis (in social media analysis, to identify positive and negative customer sentiments).
- **Recommendation System:** Naive Bayes Classifier and Collaborative Filtering together builds a Recommendation System that uses machine learning and data mining techniques to filter unseen information and predict whether a user would like a given resource or not.

Machine Learning Algorithms

- Decision Tree
- Naïve Bayes
- **Linear Regression**
- Logistic Regression
- Support Vector Machines

Linear Regression

- Linear regression is one of the easiest and most popular Machine Learning algorithms.
- It is a statistical method that is used for predictive analysis.
- Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price,** etc.
- Linear regression algorithm shows a linear relationship between a dependent (y) variable and one or more independent (x) variables, hence called as **linear regression**.
- Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

Linear Regression

- The linear regression model provides a sloped straight line representing the relationship between the variables.
- Consider the image.
- Mathematically, a linear regression is represented as:

$$Y=a_0+a_1X+\varepsilon$$

- Here,

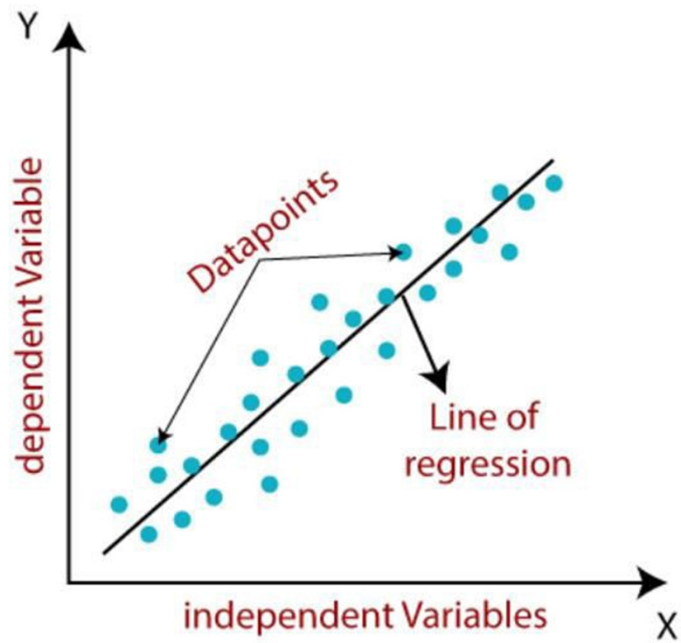
Y= Dependent Variable (Target Variable)

X= Independent Variable (predictor Variable)

a0= intercept of the line (Gives an additional degree of freedom)

a1 = Linear regression coefficient (scale factor to each input value).

ε = random error

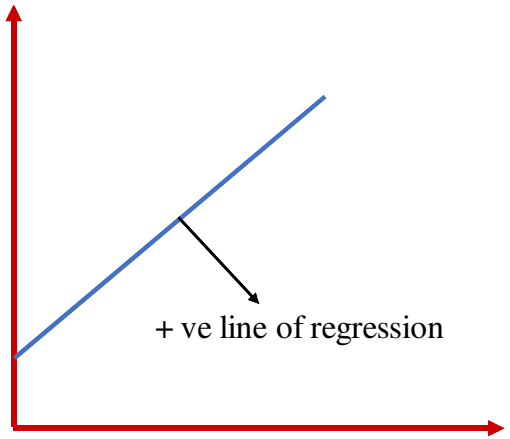


Linear Regression Line

- A linear line showing the relationship between the dependent and independent variables is called a **regression line**.
- A regression line can show two types of relationship:
- **Positive Linear Relationship:**
  If the dependent variable increases on the Y-axis and independent variable increases on X-axis, then such a relationship is termed as a Positive linear relationship.
- **Negative Linear Relationship:**
  If the dependent variable decreases on the Y-axis and independent variable increases on the X-axis, then such a relationship is called a negative linear relationship.

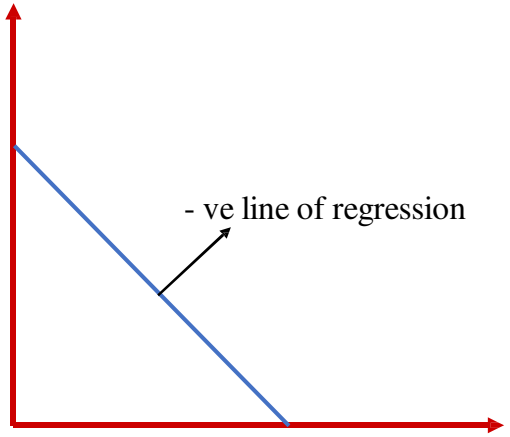
Linear Regression Line

- **Positive Linear Relationship:**



The line Equation will be: $Y= a_0+a_1x$

- **Negative Linear Relationship:**



The line Equation will be: $Y= -a_0+a_1x$

Example: Making Predictions with Linear Regression

- Given the representation is a linear equation, making predictions is as simple as solving the equation for a specific set of inputs.
- Imagine we are predicting weight (y) from height (x).
- A linear regression model representation for this problem would be:

$$\mathbf{Y = b_0 + b_1 X}$$

or

$$\mathbf{weight = b_0 + b_1 * height}$$

Example: Making Predictions with Linear Regression

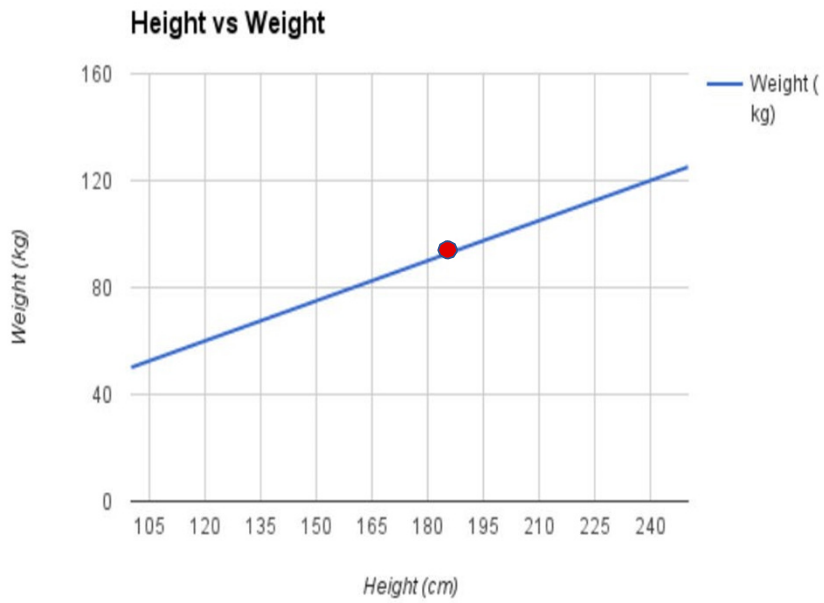
- Where $\mathbf{b_0}$ is the bias coefficient and $\mathbf{b_1}$ is the coefficient for the height column.
- A learning technique is used to find a good set of coefficient values.
- Once found, user can switch in different height values to predict the weight.
- For example, lets use $\mathbf{b_0} = 0.1$ and $\mathbf{b_1} = 0.5$.
- Let's plug them in and calculate the weight (in kilograms) for a person with the height of 182 centimeters.

$$\textbf{weight} = 0.1 + 0.5 * 182$$

$$\textbf{weight} = 91.1$$

Example: Making Predictions with Linear Regression

- The above equation could be plotted as a line in two-dimensions.
- The $\mathbf{b_0}$ is our starting point regardless of what height we have.
- We can run through a bunch of heights from 100 to 250 centimeters and plug them to the equation and get weight values, creating our line.



Preparing Data For Linear Regression

- **Linear Assumption**. Linear regression assumes that the relationship between input and output is linear. It does not support anything else. This may be obvious, but it is good to remember when we have a lot of attributes. This may need to transform data to make the relationship linear (e.g. log transform for an exponential relationship).
- **Remove Noise**. Linear regression assumes that input and output variables are not noisy. Consider using data cleaning operations that let you better expose and clarify the signal in your data. This is most important for the output variable and to remove outliers in the output variable (y) if possible.

Preparing Data For Linear Regression

- **Remove Collinearity**. Linear regression will over-fit your data when you have highly correlated input variables. Consider calculating pairwise correlations for your input data and removing the most correlated.
- **Gaussian Distributions**. Linear regression will make more reliable predictions if your input and output variables have a Gaussian distribution. You may get some benefit using transforms on you variables to make their distribution more Gaussian looking.
- **Rescale Inputs**: Linear regression will often make more reliable predictions if you rescale input variables using standardization or normalization.

Types of Linear Regression

- Linear regression can be further divided into two types of the algorithm:
  - Simple Linear Regression
  - Multiple Linear Regression

Simple Linear Regression

- If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.
- The key point in Simple Linear Regression is that the ***dependent variable must be a continuous/real value***.
- However, the independent variable can be measured on continuous or categorical values.
- The Simple Linear Regression model can be represented using the below equation:

$$Y = a_0 + a_1 x + \varepsilon$$

Multiple Linear regression:

- If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.
- In Multiple Linear Regression, the dependent variable(Y) is a linear combination of multiple independent variables $x_1, x_2, x_3, ...,x_n$.
- Since it is an enhancement of Simple Linear Regression, so the same is applied for the multiple linear regression equation, the equation becomes:

$$Y= a_0+a_1x_1+ a_2x_{2+} a_3x_{3+}.....+ a_nx_n$$

- Where,
- Y= dependent variable
- $b_0, b_1, b_2, b_3 , b_n....$= Coefficients of the model.
- $x_1, x_2, x_3, x_4,...$= Various Independent/feature variable

Advantages and Disadvantages of Linear Regression

Advantages	Disadvantages
Linear regression performs exceptionally well for linearly separable data.	The assumption of linearity between dependent and independent variables.
Easier to implement, interpret and efficient to train.	It is often quite prone to noise and overfitting.
It handles overfitting pretty well using dimensionally reduction techniques, regularization, and cross-validation.	Linear regression is quite sensitive to outliers.
One more advantage is the extrapolation beyond a specific data set	It is prone to multicollinearity

Applications of Linear Regression

- Sales Forecasting
- Risk Analysis
- Housing Applications - To Predict the prices and other factors
- Finance Applications- To Predict Stock prices, investment evaluation, etc.

Machine Learning Algorithms

- Decision Tree
- Naïve Bayes
- Linear Regression
- Logistic Regression
- Support Vector Machines

Logistic Regression

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique.
- It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable.
- Therefore the outcome must be a categorical or discrete value.
- It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1**.

Logistic Regression

- Logistic Regression is much similar to the Linear Regression except that how they are used.
- Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems**.
- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a person is obese or not based on his weight, etc.

Logistic Regression

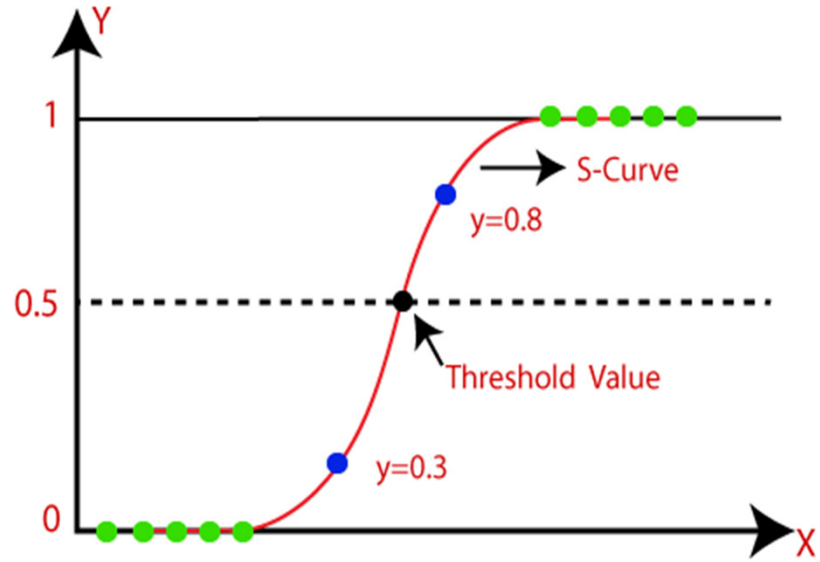
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification.

Logistic Regression

- The below image is showing the logistic function:

•**Prediction < 0.5 → Class 0**

•**Prediction >= 0.5 →Class 1**



Logistic Function (Sigmoid Function):

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

Assumptions for Logistic Regression:

- The dependent variable must be categorical in nature.
- The independent variable should not have multi-collinearity.

***Note**: Logistic regression uses the concept of predictive modeling as regression; therefore, it is called logistic regression, but is used to classify samples; Therefore, it falls under the classification algorithm.*

Logistic Regression Equation:

- The Logistic regression equation can be obtained from the Linear Regression equation.
- The mathematical steps to get Logistic Regression equations are given below:
- We know the equation of the straight line can be written as:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \ldots\ldots\ldots + b_nx_n$$

- In Logistic Regression ***y*** can be between 0 and 1 only, so for this let's divide the above equation by (1-y):

$$\frac{y}{1-y}$$

0 for y=0, and infinity for y=1

Logistic Regression Equation:

- But we need range between -[infinity] to +[infinity], then take logarithm of the equation it will become:

$$\log [y/1-y] = b_0 + b_1 x_1 + b_2 x_2 + b_3 x_3 + \ldots\ldots\ldots + b_n x_n$$

- The above equation is the final equation for Logistic Regression.

Types of Logistic Regression:

- On the basis of the categories, Logistic Regression can be classified into three types:

1. **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.

2. Multinomial: In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep“

3. **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

Applications of Logistic Regression

• Spam Detection

- Spam detection is a binary classification problem where we are given an email and we need to classify whether or not it is spam.
- If the email is spam, we label it 1; if it is not spam, we label it 0.
- In order to apply Logistic Regression to the spam detection problem, the following features of the email are extracted: Sender of the email, Number of types in the email, Occurrence of words/phrases like "offer", "prize", "free gift", etc.
- The resulting feature vector is then used to train a Logistic classifier which emits a score in the range 0 to 1. If the score is more than 0.5, we label the email as spam. Otherwise, we don't label it as spam.

- **Credit Card Fraud Detection**

- In banking sector when a credit card transaction happens, the bank makes a note of several factors. For instance, the date of the transaction, amount, place, type of purchase, etc. Based on these factors, they develop a Logistic Regression model of whether or not the transaction is a fraud. For instance, if the amount is too high and the bank knows that the concerned person never makes purchases that high, they may label it as a fraud.

- **Tumour Prediction**

- A Logistic Regression classifier may be used to identify whether a tumour is malignant or if it is benign. Several medical imaging techniques are used to extract various features of tumours. For instance, the size of the tumour, the affected body area, etc. These features are then fed to a Logistic Regression classifier to identify if the tumour is malignant or if it is benign.

Marketing

- Every day, when you browse your Facebook newsfeed, the powerful algorithms running behind the scene predict whether or not you would be interested in certain content (which could be, for instance, an advertisement).
- Such algorithms can be viewed as complex variations of Logistic Regression algorithms where the question to be answered is simple – will the user like this particular advertisement in his/her news feed?

Machine Learning Algorithms

- Decision Tree
- Naïve Bayes
- Linear Regression
- Logistic Regression
- Support Vector Machines

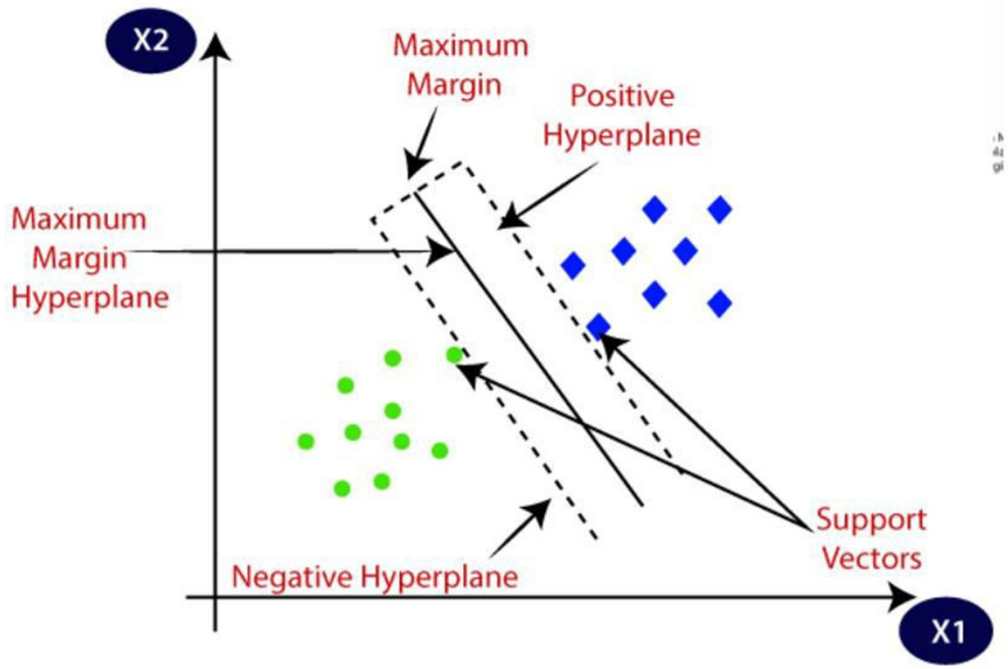
Support Vector Machines

- Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems.
- However, primarily, it is used for Classification problems in Machine Learning.
- SVMs have their unique way of implementation as compared to other machine learning algorithms.
- Lately, they are extremely popular because of their ability to handle multiple continuous and categorical variables.
- SVM algorithm can be used for **Face detection, image classification, text categorization,** etc.

Support Vector Machines

- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that one can easily put the new data point in the correct category in the future.
- This best decision boundary is called a **hyperplane**.
- SVM chooses the extreme points/vectors that help in creating the hyperplane.
- These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine.

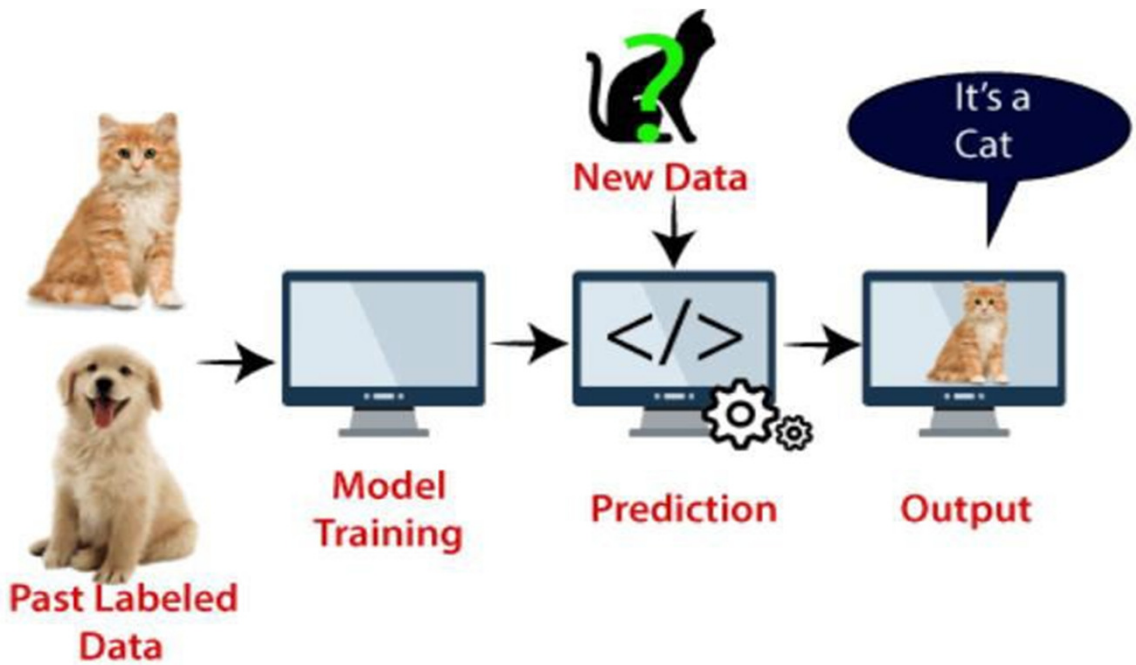
- Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example

- Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm.
- We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature.
- So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog.
- On the basis of the support vectors, it will classify it as a cat.

Consider the below diagram:



Types of Support Vector Machines

- **Linear SVM:**
- Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:**
- Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

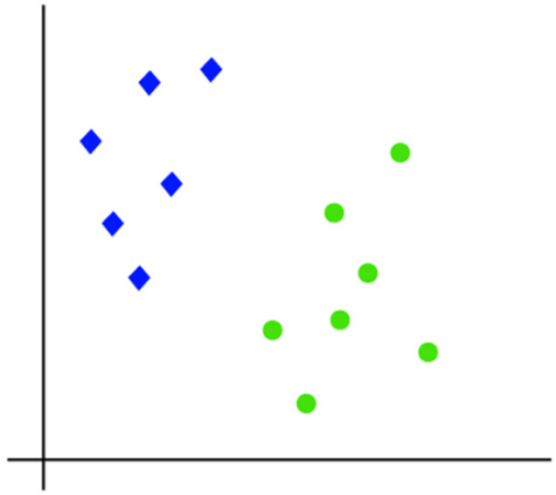
Hyperplane & Support Vectors in the SVM :

- **Hyperplane:** There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM. The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane. We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.
- **Support Vectors:** The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. These vectors support the hyperplane, hence called a Support vector

How does SVM works?

- **Linear SVM:**
- The working of the SVM algorithm is shown using an example.
- Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x1 and x2.
- We want a classifier that can classify the pair(x1, x2) of coordinates in either green or blue.

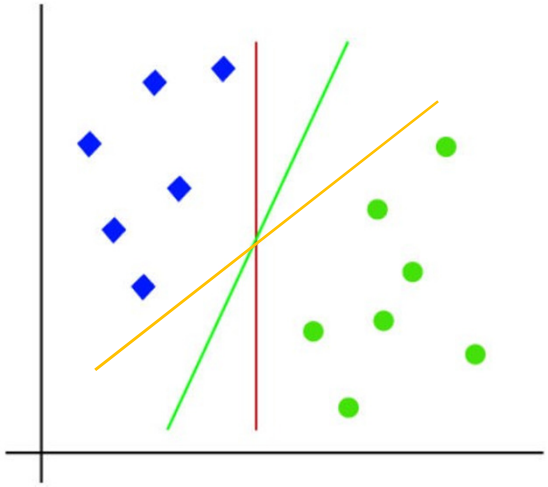
Consider the below image:



How does SVM works?

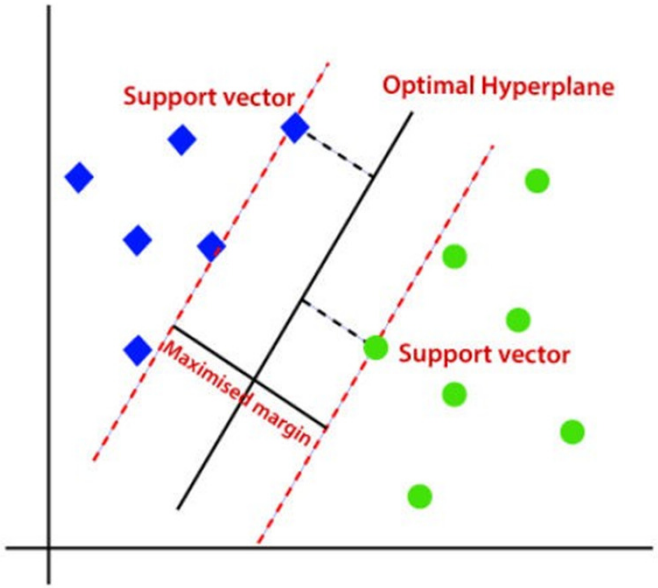
- So as it is 2-d space so by just using a straight line, two classes can be easily separated.
- But there can be multiple lines that can separate these classes.

Consider the below image:



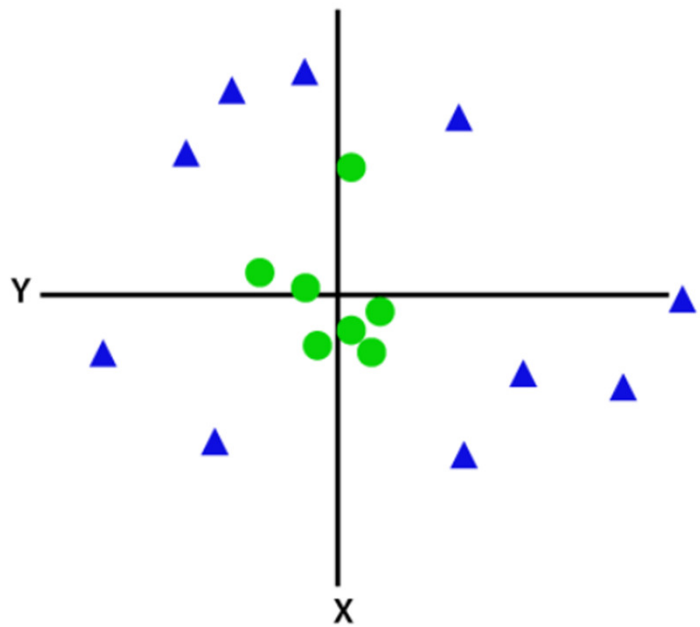
How does SVM works?

- Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**.
- SVM algorithm finds the closest point of the lines from both the classes. These points are called **support vectors**.
- The distance between the vectors and the hyperplane is called as **margin**.
- And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



How does SVM works?

- **Non-Linear SVM:**
- If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the image:

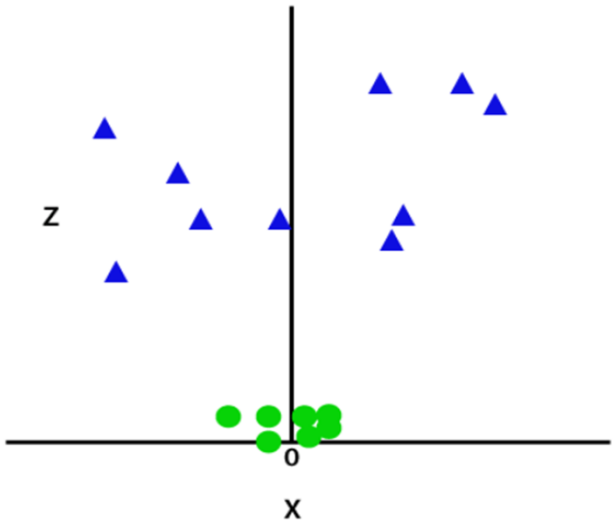


How does SVM works?

- So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z.
- It can be calculated as:

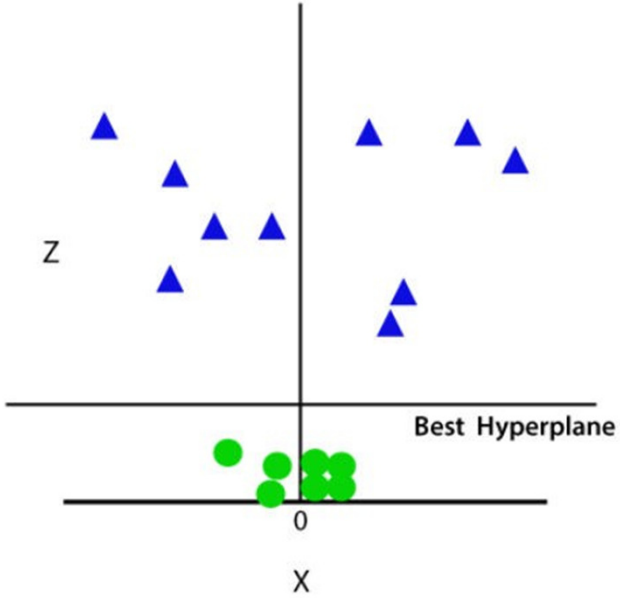
$$z = x^2 + y^2$$

By adding the third dimension, the sample space will become as below image:



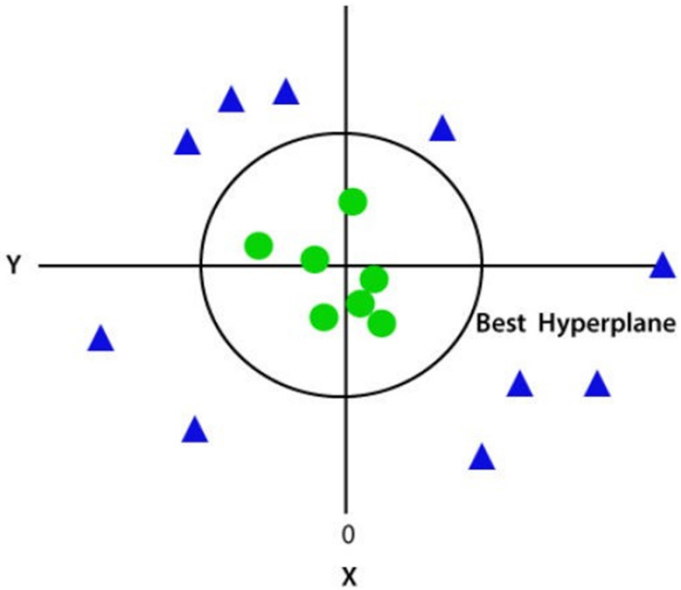
How does SVM works?

- So now, SVM will divide the datasets into classes in the following way.
- Consider the below image:



How does SVM works?

- Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with z=1, then it will become as:
- Hence we get a circumference of radius 1 in case of non-linear data.



SVM Kernels

- The SVM algorithm is implemented with kernel that transforms an input data space into the required form.
- SVM uses a technique called the kernel trick in which kernel takes a low dimensional input space and transforms it into a higher dimensional space.
- In simple words, kernel converts non-separable problems into separable problems by adding more dimensions to it.
- It makes SVM more powerful, flexible and accurate.
- The following are some of the types of kernels used by SVM.
  - **Linear Kernel**
  - **Polynomial Kernel**
  - **Radial Basis Function(RBF) Kernel**

SVM Kernels

- **Linear Kernel**
- It can be used as a dot product between any two observations. The formula of linear kernel is as below:

$$K(x\ ,\ x_i)=sum(x*x_i)$$

- From the above formula, we can see that the product between two vectors say x & xi is the sum of the multiplication of each pair of input values.

- **Polynomial Kernel**
- It is more generalized form of linear kernel and distinguish curved or nonlinear input space. Following is the formula for polynomial kernel –

$$K(x\ ,\ x_i)=\ 1+sum(x\ ,\ x_i)\text{^}d$$

- Here d is the degree of polynomial, which we need to specify manually in the learning algorithm.

SVM Kernels

- **Radial Basis Function (RBF) Kernel**

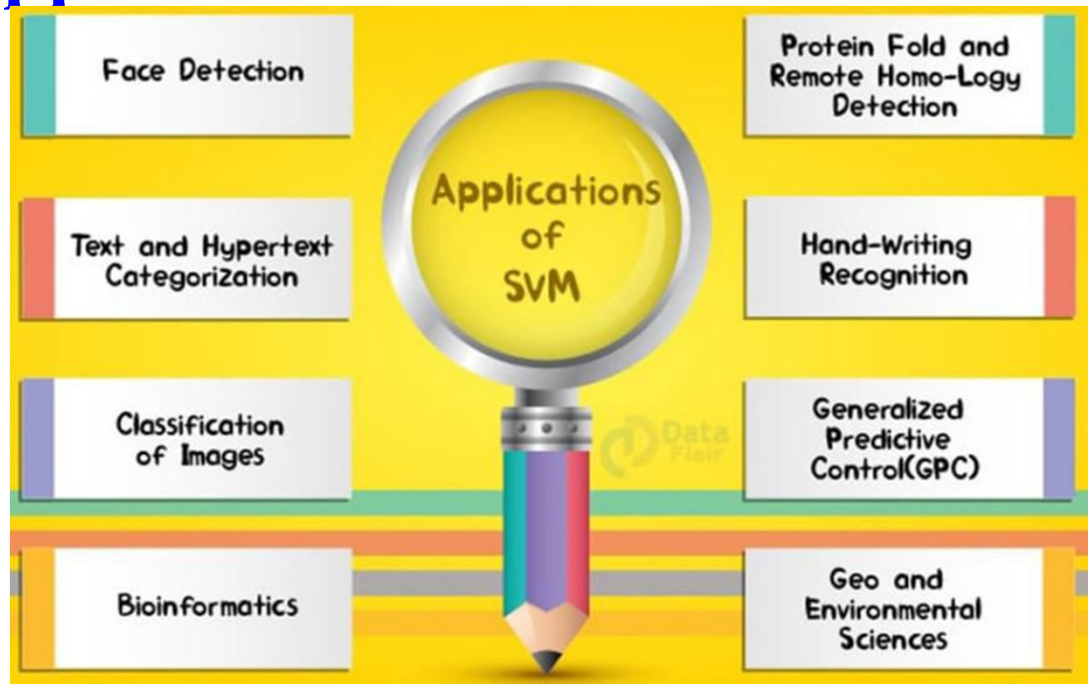
RBF kernel, mostly used in SVM classification, maps input space in indefinite dimensional space. Following formula explains it mathematically –

$$K(x , x_i) = exp(-\gamma \| x - x_i \|^2)$$

Here, *gamma* ranges from 0 to 1. We need to manually specify it in the learning algorithm. A good default value of *gamma* is 0.1.

- **Advantages of SVM**
  - It works really well with a clear margin of separation.
  - It is effective in high dimensional spaces.
  - It is effective in cases where the number of dimensions is greater than the number of samples.
  - It uses a subset of training points in the decision function (called support vectors), so it is also memory efficient.
  - SVM Classifiers offer good accuracy and perform faster prediction compared to other Machine Learning models.
- **Disadvantages of SVM**
  - SVM is not suitable for large datasets because of its high training time and it also takes more time in training.
  - It also doesn't perform very well, when the target classes are overlapped.

- **Applications of SVM**



Beyond binary classifications: multiclass classification

- **Binary Classifiers for Multi-Class Classification**
- Classification is a predictive modeling problem that involves assigning a class label to an example.
- Binary classification are those tasks where examples are assigned exactly one of two classes.
- Multi-class classification is those tasks where examples are assigned exactly one of more than two classes:
  - **Binary Classification**: Classification tasks with two classes.
  - **Multi-class Classification**: Classification tasks with more than two classes.

Beyond binary classifications: multiclass classification

- One approach for using binary classification algorithms for multi-classification problems is to split the multi-class classification dataset into multiple binary classification datasets and fit a binary classification model on each.
- Two different methods of this approach are the **One-vs-Rest** and **One-vs-One** strategies.
- The **One-vs-Rest** strategy splits a multi-class classification into one binary classification problem per class.
- The **One-vs-One** strategy splits a multi-class classification into one binary classification problem per each pair of classes.

One-Vs-Rest for Multi-Class Classification

- One-vs-rest (OvR for short, also referred to as One-vs-All or OvA) is a heuristic method for using binary classification algorithms for multi-class classification.

- It involves splitting the multi-class dataset into multiple binary classification problems. A binary classifier is then trained on each binary classification problem and predictions are made using the model that is the most confident.
- For example, given a multi-class classification problem with examples for each class '*red*,' '*blue*,' and '*green*'. This could be divided into three binary classification datasets as follows:

- **Binary Classification Problem 1**: red vs [blue, green]
- **Binary Classification Problem 2**: blue vs [red, green]
- **Binary Classification Problem 3**: green vs [red, blue]

One-Vs-Rest for Multi-Class Classification

- A possible downside of this approach is that it requires one model to be created for each class. For example, three classes requires three models. This could be an issue for large datasets (e.g. millions of rows), slow models (e.g. neural networks), or very large numbers of classes (e.g. hundreds of classes).
- *"The obvious approach is to use a one-versus-the-rest approach (also called one-vs-all), in which we train C binary classifiers, fc(x), where the data from class c is treated as positive, and the data from all the other classes is treated as negative."*

One-Vs-One for Multi-Class Classification

- One-vs-One (OvO for short) is another heuristic method for using binary classification algorithms for multi-class classification.
- Like one-vs-rest, one-vs-one splits a multi-class classification dataset into binary classification problems.
- Unlike one-vs-rest that splits it into one binary dataset for each class, the one-vs-one approach splits the dataset into one dataset for each class versus every other class.

One-Vs-One for Multi-Class Classification

- For example, consider a multi-class classification problem with four classes: '*red*,' '*blue*,' and '*green*,' '*yellow*.' This could be divided into six binary classification datasets as follows:
- **Binary Classification Problem 1**: red vs. blue
- **Binary Classification Problem 2**: red vs. green
- **Binary Classification Problem 3**: red vs. yellow
- **Binary Classification Problem 4**: blue vs. green
- **Binary Classification Problem 5**: blue vs. yellow
- **Binary Classification Problem 6**: green vs. yellow

One-Vs-One for Multi-Class Classification

- The formula for calculating the number of binary datasets, and in turn, models, is as follows: (NumClasses * (NumClasses – 1)) / 2
- We can see that for four classes, this gives us the expected value of six binary classification problems:

  (NumClasses * (NumClasses – 1)) / 2

  (4 * (4 – 1)) / 2

  (4 * 3) / 2

  12 / 2

  6

One-Vs-One for Multi-Class Classification

- Each binary classification model may predict one class label and the model with the most predictions or votes is predicted by the one-vs-one strategy.
- *"An alternative is to introduce $K(K-1)/2$ binary discriminant functions, one for every possible pair of classes. This is known as a one-versus-one classifier. Each point is then classified according to a majority vote amongst the discriminant functions."*

END
Of
UNIT- II